

Voice & Chat Agent Configuration Guide

A complete reference for configurators and IT vendors

EkStep · April 2026 ·

 For Configurators Product managers, domain experts, or anyone defining what an agent should do. Start with Part 1, then go to your use case config sheet.	 For IT Vendors Developers and technical teams building or integrating the agent. Parts 2–4 are yours. The config sheets tell you exactly what to wire up.
---	---

Document Map

Part	Title	For	What you get
Part 1	Use Case Selector	Both	Identify your agent type in 3 questions. Go to your config sheet.
Part 2	Prompt Configuration	Both	Every configurable section explained — what it is, when to use it, how to write it.
Part 3	System Configuration	IT Vendor	Tools, knowledge bases, exception handling, call vs chat design.
Part 4	Meta-Prompt Generator	Both	Questionnaire + Claude prompt that generates a production-ready agent prompt.
Config Sheets (separate)	One per use case type	Both	Single-page checklist: every decision pre-made for your agent type.

Part 1 — Use Case Selector

Before configuring anything, identify your agent type. This determines which config sheet to use and which parts of this guide apply to you.

The Four Agent Types

Transactional	Informational	Agentic	Conversational
Completes one defined task per session	Answers questions from a knowledge source	Takes multi-step actions across systems	Navigates complex, stateful, emotional journeys
Appointment booking Utility bill payment Delivery status Form submission	Scheme eligibility Policy FAQ Product info Document guidance	Job application (KKB) Lead qualification Onboarding flow Multi-system lookup	Job market advisory (KKB deep) Business advisory (DKB) Counselling Crisis support
3–6 turn conversation Fixed slot-fill logic One clear success state	No actions taken Knowledge scope is fixed Out-of-scope redirect	Tool calls required Consent before action Error recovery built in	State model required Emotional handling required Multi-session memory Dignity safety checks

Three-Question Selector

1	Does the agent take any action — an API call, form submission, or system write? NO → Go to Q2 YES → Go to Q3
2	Does it answer questions from a defined knowledge source? YES → Informational Agent → Use Config Sheet B NO → Reconsider scope. A passive listener is not an agent.
3	Is the task a single defined flow (book / check / submit) with a clear end state? YES → Transactional Agent → Use Config Sheet A NO → Go to Q4

4

Does the agent need to hold context across turns, navigate trade-offs, or respond to emotional state?

YES → Conversational Agent → Use Config Sheet D

NO → Agentic Agent → Use Config Sheet C

Note: Most real agents are a blend. Use the dominant interaction type to pick your primary config sheet, then pull sections from others as needed.

Part 2 — Prompt Configuration

This part explains every configurable prompt section — what it does, when it's needed, and how to write it. Your use case config sheet tells you which sections apply to you and in what order to arrange them.

Section	Name	Required for
§2.1	Agent Identity & Persona	All agent types
§2.2	Task Scope & Boundaries	All agent types
§2.3	Conversation Opening Logic	Agentic + Conversational
§2.4	User Universe	Conversational (required) · Agentic (optional)
§2.5	Conversation State Model	Conversational only
§2.6	Tool Invocation Rules	Transactional + Agentic + Conversational
§2.7	Knowledge Base Usage Rules	Informational + any agent with a KB attached
§2.8	Data Gathering Rules	Transactional + Agentic + Conversational
§2.9	Response Format & Presentation	All agent types
§2.10	Language & TTS Rules	All voice agents
§2.11	Style & Prohibited Language	All agent types
§2.12	Emotional & Dignity Handling	Conversational only
§2.13	Special Journey Patterns	Agentic + Conversational
§2.14	Action & Consent Rules	Transactional + Agentic + Conversational
§2.15	Graceful Exit	All agent types

§2.1 — Agent Identity & Persona

The opening block of every prompt. Sets the voice, values, and character of the agent. Everything downstream flows from this.

- **Name:**
- **Metaphor:**
- **Positive attributes:**
- **NOT statements:**
- **Core belief:**
- **For voice:**

Template

You are [NAME] — [METAPHOR].

You sound: [ATTR 1], [ATTR 2], [ATTR 3], [ATTR 4].

You are NOT: [NOT 1], [NOT 2], [NOT 3].

Core belief: [ONE SENTENCE].

§2.2 — Task Scope & Boundaries

Defines what the agent does and what it explicitly does not do. Without this, agents drift into out-of-scope behaviour that erodes user trust and creates support load.

- Primary task: one sentence, specific.
- Out-of-scope list: be explicit, not aspirational. If it's not on this list, the agent may try to handle it.
- Out-of-scope behaviour: what the agent says when it hits a boundary — redirect, not refusal. Never a lecture.

Template

Primary task: [ONE SENTENCE].

Out of scope: [LIST]. When a user asks about these, say: '[REDIRECT PHRASE — one line, natural, in target language]'.

The agent must never: [HARD PROHIBITIONS — actions, not topics].

§2.3 — Conversation Opening Logic

For Transactional and Informational agents, the opening can be a simple fixed greeting. For Agentic and Conversational agents, it must branch based on what is already known about the user — from contact memory, session history, or prior actions.

- New user, no history → open discovery opener.
- Returning user mid-journey → resume from last known point. Do NOT restart.
- Returning user post-action → acknowledge the action first before anything else.
- User with partial profile (location known, intent unknown) → use what's known, ask only what's missing.

IT Vendor note: The agent receives contact memory as an injected variable at call start (e.g. {contact_memory}). This variable must be populated by the platform before the prompt runs. If it is empty, the agent falls back to the new-user opener. Define the memory schema before writing the opening logic.

§2.4 — User Universe

Required for Conversational agents. Optional for Agentic agents where user type meaningfully changes the conversation path. Skip for Transactional and Informational.

Define the realistic range of users — not as a fixed decision tree, but as background context the agent uses to calibrate tone, detail level, and trade-off framing. The agent infers; it never labels the user aloud.

- List 3–6 user archetypes in plain language.
- For each: one sentence on what they care about most.
- Instruction to agent: 'Infer gradually from what the user says. Never assume a persona too early.'

§2.5 — Conversation State Model

Conversational agents only. This is the section that separates a scripted bot from a true conversational agent. It defines where the user is — cognitively and emotionally — and what the agent's job is in each state.

States are not a linear funnel. Users move between them in any direction. The agent detects state from what the user says, not from turn count.

State	What triggers it	Agent's primary job	What NOT to do
Fog	Vague requests, confusion, 'kuch samajh nahi'	Orient gently. Offer 2–3 directions. No interrogation.	Don't ask 5 questions. Don't jump to solutions.

State	What triggers it	Agent's primary job	What NOT to do
Orientation	Asking about options, pay, process, timelines	Show real picture. Fill only what's missing for the next useful answer.	Don't over-detail. Don't push toward a decision.
Evaluation	Comparing paths, asking about trade-offs	Surface trade-offs honestly. Never push one option as best.	Don't hide downsides. Don't create false urgency.
Commitment	'Kar do', 'theek hai', 'book karo', clear intent	Remove friction. Confirm consent. Execute.	Don't re-explain. Don't add new options.
Follow-through	Returning after action, complaint, life change	Continue journey. Acknowledge first. Never restart.	Don't defend. Don't restart from introduction.

§2.6 — Tool Invocation Rules

This section tells the agent when to call a tool, what data it needs first, and what must never substitute a tool call. It does not define the tool itself — that is done separately in your system configuration (Part 3).

- For each tool: name, trigger condition, minimum required data, and fallback behaviour on failure or empty result.
- State what MUST NOT substitute a tool call. Memory, prior context, and assumed knowledge are never valid substitutes for live data.
- Define a conversational bridge the agent can use before the tool call — one short natural line, not a 'please wait' message.
- Define emergency mode: what gets skipped when urgency is explicit and the minimum viable data is all that's needed.

Template — per tool

Tool name: [NAME]

Call when: [EXACT CONDITION — be specific, not categorical]

Required before calling: [DATA FIELDS]

Must NOT substitute this call: [e.g. memory, prior conversation, assumed availability]

If result is empty: [WHAT AGENT SAYS]

If tool fails: [WHAT AGENT SAYS]

Conversational bridge: '[ONE LINE IN TARGET LANGUAGE]'

IT Vendor note: Tool schemas and API contracts are defined outside this prompt. This section only governs the agent's decision to call, not the implementation. Ensure tool names in this section exactly match the function names registered in your agent runtime.

§2.7 — Knowledge Base Usage Rules

Required for Informational agents. Also applies to any agent with a KB attached for factual grounding (e.g. a scheme eligibility agent or a business advisory bot).

- Define the scope of the knowledge base — what it contains and what it does not.
- Define confidence rules: what the agent does when the KB has partial information, conflicting information, or no information.
- Define citation behaviour: does the agent cite its source? For formal or regulated contexts (government, health), citation builds trust. For conversational agents, it can feel bureaucratic.
- Define the KB-to-agent boundary: the agent interprets and speaks the information naturally. It does not read out KB entries verbatim.

Confidence rule template

If KB has a clear answer: state it in natural spoken language.

If KB has partial information: share what is known, acknowledge the gap. Do not guess.

If KB has no information: 'Iske baare mein mujhe pakki jaankari nahi hai.' Do not fabricate.

If KB answers conflict: surface the conflict honestly. Do not pick one without basis.

§2.8 — Data Gathering Rules

Applies wherever the agent needs information from the user before it can act. The core discipline: only ask what changes the next useful answer.

- Define the Minimum Viable Data Set (MVDS) — the smallest set of information needed to take the next step.
- Never ask for everything upfront. Never ask more than one question per turn.
- Define refinement fields — information that's useful but only needed after the first useful result is shown.
- Define emergency mode — what fields are skipped when same-day urgency is explicit.
- Do not ask for information the system already has from contact memory or prior session.

Agent Type	Minimum Viable Data Set	Refinement Fields (ask after first result)
Transactional	All required slots for the specific action (e.g. name + date + department for a booking)	Preferences that modify the result (e.g. morning vs afternoon)

Agent Type	Minimum Viable Data Set	Refinement Fields (ask after first result)
Informational	The specific question or topic	Context that changes the answer (e.g. state, income bracket)
Agentic	Location + intent/role/task category	Income range, commute limit, urgency, qualification
Conversational	Intent category + one grounding detail	Everything else — gathered through natural conversation flow

§2.9 — Response Format & Presentation Rules

Defines how information is presented when results are returned — especially for tool results or KB lookups. Prevents information dumps and menu-style output.

- Short-list rule: never present more than 3 options at once. Always end with a question.
- Deep-dive rule: only go into detail on the option the user has selected or asked about.
- Salary / data normalisation: always present ranges in correct order (low to high). Never expose raw API fields (IDs, GPS, internal scores).
- Progressive disclosure: in early turns, give category + range + one differentiator. In later turns, go deeper only on request.

Short-list presentation template (voice)

'[N] options hain —

Pehla: [ROLE], [EMPLOYER], [CITY], [SALARY RANGE].

Doosra: [ROLE], [EMPLOYER], [CITY], [SALARY RANGE].

Teesra: [ROLE], [EMPLOYER], [CITY], [SALARY RANGE].

Koi sawaal hai? Ya kisi ek ke baare mein aur jaanna chahenge?'

§2.10 — Language & TTS Rules

Required for all voice agents. This is the most commonly missed section and the most consequential. TTS engines do not reliably normalise numbers, symbols, abbreviations, or mixed-script text. Every data type must have an explicit rule.

Data Type	What TTS does if you skip this rule	What to specify in the prompt
Numbers	'1500' → 'one comma five hundred' or spoken as digit string	Write in words. '1,500' → 'paanch sau' / 'fifteen hundred'
Money	'₹13,000' → 'rupee symbol thirteen comma zero zero zero'	Always in words. '₹13,000' → 'terah hazaar rupaye'
Date	'29/01/2026' → 'twenty-nine slash one slash twenty twenty-six'	Expanded spoken form. 'unnatees January do hazaar chhabbees'
Time	'3 PM' → 'three P M' or skipped	Spoken form only. 'dopahar teen baje'. Never AM/PM for Hindi.
Phone	'9876543210' → spoken as a large number	Digit by digit in words. 'nau, aath, saat, chhah...'
Abbreviations	'ITI' → 'iti' or mispronounced	Expand as spoken letters. 'Aai Tee Aai'
Output script (Hindi)	Roman Hindi in output → broken pronunciation	Devanagari script only. No Roman Hindi in output. Ever.
English loanwords	Depends on TTS engine — often mispronounced	Provide Devanagari transliteration. 'job' → 'जॉब'

§2.11 — Style & Prohibited Language

Defines what the agent sounds like and what it must never say. The prohibited list is often more powerful than positive instructions — it catches edge cases that style guidance misses.

- Write style as observable output behaviours, not abstract qualities.
- Prohibited phrases must be specific. 'Don't worry', 'sab theek ho jaayega', 'guaranteed' — not just 'avoid sales language'.
- Prohibited patterns cover structural anti-patterns: multiple questions in one turn, menu-style prompts, reading back all captured data at once.
- Include a pre-response dignity check for Conversational agents: Does this blame the user? Does it over-promise? Does it reduce their agency? Does it sound like a script?

§2.12 — Emotional & Dignity Handling

Conversational agents only. Defines how the agent responds to emotional signals — distress, frustration, disappointment, overwhelm — without coaching, pitying, or applying pressure.

- Acknowledge emotion without amplifying it. Mirror at low intensity.
- Never use motivational language as a response to distress.
- Silence after bad news: let the truth land. One gentle bridge maximum. No new questions immediately.
- If emotion is high: reduce complexity, not increase it. Fewer options, simpler language.

Situation	Allowed	Never
User is disappointed	'Samajh mein aata hai.' / 'Haan, yeh nirash karne wala lag sakta hai.'	'Don't worry.' / 'Sab theek hoga.' / 'Aap kar lenge.'
User is overwhelmed	Pause. Simplify. One thing at a time.	Flood with more options. Ask multiple questions.
User is angry	Acknowledge first. Never defend.	Explain or justify. Escalate tone.
User is silent	Short pause: wait. Long pause: one bridge — 'Koi baat nahi, sochiye.'	Fill silence with questions. Repeat last message.

§2.13 — Special Journey Patterns

Applies to Agentic and Conversational agents. Defines explicit behaviour for edge cases that are predictable but easy to miss. Each pattern needs its own handling rule in the prompt.

Pattern	Trigger	What the agent must do
Proxy caller	Caller is acting on behalf of someone else	Identify the actual candidate. Gather info about them. Keep the path easy for that person to continue later.
Emergency mode	User needs help today or very soon	Compress to minimum viable data only. Skip all non-essential profiling. Act immediately.
Repeated indecision	User has reviewed options across sessions but can't commit	Probe gently for an external blocker. Don't increase pressure. Offer human escalation as support, not exit.
Do-not-call request	User asks not to be contacted again	Comply immediately. No persuasion. No final pitch. No guilt.
Complaint / mismatch	Something didn't match what was promised	Acknowledge first. Never defend. Understand what changed. Reopen the journey.

Pattern	Trigger	What the agent must do
Mid-journey drop and return	User dropped a previous session	Resume from last usable point. Do not restart from introduction.
Post-action return	User returns after applying, booking, or enrolling	Continue. Check: did it work? What changed? Never restart.

§2.14 — Action & Consent Rules

Any agent that takes an action on behalf of the user must ask for explicit verbal consent before doing so. This applies to all agent types that take actions. Never infer readiness from conversational context alone.

- List every action the agent can take.
- For each action: the exact consent phrase in the target language.
- Prohibited: pressure language, urgency framing, loss framing ('this slot will go').
- If consent is declined: acknowledge, offer alternatives, never push.

Consent phrase template

Action: [WHAT THE AGENT DOES]

Consent phrase: '[NATURAL QUESTION IN TARGET LANGUAGE — e.g. Kya main aapki taraf se apply kar doon?']

On decline: '[GRACEFUL RESPONSE — offer alternative or simply acknowledge]'

Never say: '[PRESSURE PHRASE — e.g. yeh mauka chala jaayega / abhi decide kijiye]'

§2.15 — Graceful Exit

Required for all agent types. Defines how the conversation ends cleanly, with closure, without pressure.

- Confirm nothing else is needed before closing.
- One-line natural recap of what was discussed or completed.
- For voice: a fixed terminal word that signals call end to the telephony system. This word must be consistent across all sessions. Choose one and lock it in.
- For chat: a warm but brief sign-off. No fixed token needed.
- Never end on a question that restarts the conversation.

IT Vendor note: The terminal word (e.g. 'Goodbye') is the trigger for the telephony layer to end the call. Confirm with your platform what string it listens for. If the agent varies this word, calls may not terminate cleanly.

Part 3 — System Configuration

This part is primarily for IT vendors and developers. It covers the non-prompt configuration decisions that determine whether a well-written prompt actually works in production.

§3.1 — Tool Configuration

Tools are defined outside the prompt — as API endpoints, function schemas, or platform connectors. The prompt only governs when and how to invoke them. This section covers what the IT vendor needs to configure for tool invocation to work correctly.

A

Tool Registration

Every tool referenced in the prompt by name must be registered in the agent runtime with exactly the same name.

If the prompt says `get_jobs`, the function must be registered as `get_jobs` — not `getJobs`, not `fetch_jobs`.

Mismatched names cause silent failures — the agent appears to call the tool but nothing happens.

B

Input Payload Construction

The agent constructs the tool call payload from available context: contact memory, profile data, conversation history.

The prompt must specify which fields come from memory vs. which are gathered in conversation.

Optional fields must be clearly distinguished from required fields in the tool schema — the agent must not send empty strings for optional fields.

C

Response Handling

Define how the agent interprets the tool response: which fields to speak, which to suppress, and in what order.

Never expose: internal IDs, GPS coordinates, raw match scores, hiring manager contact details.

Always validate: `is_active` / `status` fields before presenting any result. Never present inactive or closed results.

D

Failure Modes

Tool timeout: agent must acknowledge and offer to try again or take an alternative path.

Empty result: agent must say so honestly in natural language. It must not fabricate results.

Partial result: agent presents what exists, acknowledges what's missing.

Auth failure: agent must not surface error codes to the user. Acknowledge and suggest human escalation.

Testing requirement: Every tool invocation path in the prompt must be tested with (a) a successful response, (b) an empty response, and (c) a failure response before going live.

§3.2 — Knowledge Base Configuration

For agents with a KB attached — Informational agents primarily, but also any Agentic or Conversational agent with factual grounding needs.

A KB Scope Definition

The KB must have a clear, documented scope — what it contains, what version it is, and when it was last updated.

The agent must know the scope so it can answer 'I don't know' correctly for out-of-scope queries.

Avoid putting everything into a single KB. Narrow, well-scoped KBs produce better retrieval than broad ones.

B Retrieval Behaviour

Configure chunk size and overlap appropriate to how questions will be asked — typically shorter for voice (one question per turn) than chat.

Set a confidence threshold. Below it, the agent should acknowledge uncertainty rather than guess.

Test retrieval with 20–30 representative user questions before launch. Note failure modes and adjust chunking or metadata accordingly.

C KB Update Process

Define how often the KB is updated and who is responsible.

When the KB is updated, regression-test the queries that previously failed — new content can break old retrieval.

Version the KB. The agent prompt should reference which KB version it is calibrated against.

§3.3 — Contact Memory & Session State

For Agentic and Conversational agents, contact memory is the mechanism that allows the agent to resume journeys across sessions. Without it, every call starts from zero.

Memory Type	What it stores	Who writes it	Lifecycle
Session memory	Everything from the current call: intent, data gathered,	Agent runtime, auto	Cleared at end of session — persisted to contact record

Memory Type	What it stores	Who writes it	Lifecycle
	options shown, actions taken		
Contact memory	Persistent profile: location, trade, preferences, last action, last status	Platform CRM / contact record	Persists across sessions. Written by agent. Read at call start.
Profile data	Richer structured profile: age, languages, education, work history	External system (e.g. profile API)	Fetches on demand via get_profile tool, not stored in prompt
Do-not-contact flag	Explicit opt-out flag	Agent writes on user request	Permanent until user reverses

IT Vendor note: The contact memory object must be injected into the prompt at call start as a variable (e.g. {contact_memory}). The prompt's opening logic branches on the contents of this variable. If it is null or empty, the agent must treat the user as new. Define the memory schema explicitly and document it — prompt behaviour depends entirely on what fields exist.

§3.4 — Exception Handling

Exceptions are not edge cases — they are guaranteed to occur. Every exception type needs a defined behaviour in both the prompt and the system layer.

Exception	Prompt behaviour	System layer responsibility
Tool timeout (>5s)	'Ek second...' then retry once. On second failure, acknowledge and offer alternative path.	Set timeout thresholds. Implement retry with exponential backoff. Log all failures.
Empty tool result	State clearly in natural language. Do not fabricate. Offer alternative (broader search, WhatsApp follow-up).	Ensure API returns a clear empty state, not a null or error, so the agent can distinguish 'no results' from 'failed'.
KB retrieval below threshold	Acknowledge uncertainty. Do not guess. Offer to connect to a human.	Configure confidence threshold in retrieval system. Return confidence score to agent.
User goes silent (voice)	Short pause: wait. Long pause (>8s): one gentle bridge. After	Platform must handle silence detection and pass signal to agent. Define silence timeout in platform config.

Exception	Prompt behaviour	System layer responsibility
	second silence: offer to call back.	
Out-of-scope request	Redirect naturally. One line. Never lecture.	Log out-of-scope queries — they are product signals. Review weekly.
User abusive or distressed	De-escalate. Reduce complexity. If persistent: offer human escalation.	Define escalation path: does the platform support warm transfer to a human agent?
ASR misrecognition (voice)	Ask for clarification once. Natural rephrase — never 'I didn't understand'.	Tune ASR to domain vocabulary. Add custom words: trade names, place names, scheme names.
Session drops mid-call	On reconnect: resume from contact memory. Do not restart.	Ensure session state is written to contact record at every significant turn, not just at end of call.

§3.5 — Call vs. Chat Design Differences

The same agent logic requires meaningfully different configuration depending on whether the channel is voice (call) or chat (WhatsApp, web widget, in-app). This table maps every dimension where configuration differs.

Dimension	Voice (Call)	Chat
Output script	Native script only (Devanagari for Hindi). Roman Hindi breaks TTS pronunciation completely.	Any script. Code-switching is natural and fine.
Number formatting	All numbers in words. '₹13,000' → 'terah hazaar rupaye'. Digits break TTS.	Digits and symbols render correctly. '₹13,000' is clear.
Date & time	Spoken form only. 'dopahar teen baje'. Never AM/PM for Hindi.	Standard formats work. '3 PM', '29 Jan'.
Message length	One idea per turn. 2–3 sentences max. Longer = lost caller.	Short paragraphs preferred for mobile, but slightly more depth is OK.
Formatting	No markdown. No bullets. No emojis. TTS reads them literally.	Bold, short lists, emojis — use sparingly and only for clarity.

Dimension	Voice (Call)	Chat
Silence	Explicit silence detection and handling rules required.	No silence concept. User reads at their own pace.
Quick replies / menus	No menus. No option lists read aloud. One spoken direction only.	Quick reply buttons acceptable for Transactional agents. Not for Conversational.
Consent	Spoken consent phrase. Conversational register.	Can be a text question or a quick reply button.
Exit / termination	Fixed terminal word required (e.g. 'Goodbye') for telephony layer to end the call.	Natural sign-off. No terminal token needed.
Error / waiting	One short spoken bridge ('ek second'). Never 'please hold' or 'please wait'.	Typing indicator or 'Searching...' is natural.
ASR / input	Agent must handle misrecognition gracefully. Vocabulary tuning required.	User types. Typos and abbreviations common. Agent must handle gracefully.
Turn pacing	Agent controls pace. One idea at a time. Wait for response.	User controls pace. Agent can be slightly more information-dense.

Part 4 — Meta-Prompt Generator

Fill in the questionnaire below, then paste your answers into the meta-prompt and run it in Claude. The output will be a production-ready agent prompt, calibrated to your agent type and channel.

This questionnaire replaces generic intake. Your use case config sheet (separate document) already tells you which sections you need — answer questions in that context.

The Questionnaire

Answer in plain language. For each answer, more honest specificity = better output.

A — Identity

1. Agent name, and what it should feel like to talk to (one phrase).
2. 4–6 words describing how it sounds.
3. 4–6 things it must never sound like or say. Include example phrases.
4. Male / female / neutral voice? Conversational, semi-formal, or formal register?
5. One sentence: what does this agent fundamentally stand for?

B — Users

6. Who uses this agent? Describe in plain language — their situation, not demographic.
7. What is the most common reason they reach out?
8. What is the hardest situation a user might be in? What does that look like in a real call?
9. Are there meaningfully different user types? If yes, what does each care about most?

C — Task & Scope

10. The agent's one main task, in one sentence.
11. Everything explicitly out of scope. Be specific.
12. What does the agent say when a user asks for something out of scope?
13. What must the agent never do, regardless of how the user asks?

D — Tools & Data

14. What tools does the agent call? For each: what triggers the call, and what data is required before calling?
15. What is the minimum data needed before the agent can take any action?
16. What happens if a tool returns no results? What if it fails?
17. Does the agent have access to a knowledge base? What does it contain? What is out of scope for the KB?
18. What contact memory or session state is available at call start? What fields does it contain?

E — Conversation Flow

19. Walk through a typical successful conversation in plain steps.
20. What does the agent do if it doesn't have enough information yet?
21. Does the agent need to remember things across multiple sessions? What?
22. What happens at the end of a successful conversation?

F — Language & Format

23. Exact language and dialect. For voice: confirm all numbers/money/dates/times must be in words.
24. Any English words that must appear in output? How should they be written?
25. Strictly banned words or phrases — list them.

G — Edge Cases

26. Most urgent scenario — how fast does the user need help?
27. What if someone calls on behalf of another person?
28. What if the user complains about a previous interaction?
29. What if the user asks to stop being contacted?
30. Anything else the agent must always do, or must never do?

The Meta-Prompt — Voice Agent

Copy this entire block into Claude. Replace [ANSWERS] with your questionnaire answers.

```
You are an expert voice agent prompt engineer.

I am giving you answers to a product questionnaire.
Generate a complete, production-ready VOICE AGENT system prompt.

STEP 1 — Classify the agent type from the answers:
  Transactional: single defined task, 3-6 turns, clear success state
  Informational: answers questions from KB, no actions taken
  Agentic: multi-step actions, tool calls, consent required
  Conversational: stateful, emotional, multi-session, trade-off navigation

  State the classification at the top before the prompt begins.

STEP 2 — Include ONLY the sections relevant to the classified type:
  All types:      Identity · Task Scope · Language/TTS · Style/Prohibited · Exit
  Transactional: + Tool Invocation · Data Gathering · Action & Consent
  Informational: + KB Usage Rules
```

```
Agentic:      + Opening Logic · Tool Invocation · Data Gathering ·
               Action & Consent · Exception Handling · Special Patterns
Conversational: All of the above + User Universe · State Model ·
               Emotional & Dignity Handling
```

Do not add sections not listed for the type. Do not over-engineer.

STEP 3 – Apply these voice-specific output rules throughout:

- Output language in native script only (e.g. Devanagari for Hindi)
- ALL numbers in words. ALL money in words. ALL dates expanded spoken form.
- Time in spoken form only (dopahar teen baje – never 3 PM for Hindi)
- Phone numbers digit by digit in words
- Abbreviations expanded as spoken letters
- One idea per turn. 2-3 sentences maximum per response.
- No markdown. No bullet lists. No emojis in output.
- Tool call bridge: one short spoken line before each tool call. Never 'please wait' or 'kripya prateeksha karein'.
- Fixed terminal word at graceful exit – identify and lock it.

STEP 4 – For tool invocation sections:

- Name each tool exactly as it will be registered in the runtime
- State trigger condition precisely
- State minimum required data before call
- State what must NOT substitute the tool call
- State failure and empty-result behaviour

STEP 5 – For the opening logic (Agentic/Conversational):

- Write branching openers for each contact memory state:
new user · sparse profile · rich profile · returning mid-journey ·
returning post-action
- Use the contact_memory variable pattern: {contact_memory}

Output format: Clean markdown. Section headers from the building block names.
Write example phrases in the target language specified in the answers.
Keep the register voice-first throughout – no form-filling feel.

--- ANSWERS START ---

[PASTE QUESTIONNAIRE ANSWERS HERE]

--- ANSWERS END ---

The Meta-Prompt — Chat Agent

Same questionnaire. Different output rules. Use this version for WhatsApp, web widget, or in-app chat.

```
You are an expert chat agent prompt engineer.
```

```
I am giving you answers to a product questionnaire.
```

```
Generate a complete, production-ready CHAT AGENT system prompt.

STEP 1 – Classify the agent type (same logic as voice version).
State the classification at the top before the prompt begins.

STEP 2 – Include only the sections relevant to the classified type
(same selection logic as voice version).

STEP 3 – Apply these chat-specific output rules:
- Numbers, symbols, dates, and times can appear as standard text.
  The user reads – TTS normalisation is NOT needed.
- Messages should feel like a knowledgeable contact texting, not a
  helpdesk ticket or government form.
- Short paragraphs. No walls of text. Mobile-first.
- Bold and short lists are acceptable – use only for clarity.
- Emojis are acceptable – use sparingly and only to signal tone.
- Quick reply buttons acceptable for Transactional agents.
  Not appropriate for Conversational agents.
- No fixed terminal word needed. Close naturally.
- A 'Searching...' or typing indicator line is acceptable before
  tool calls.

STEP 4 – Tool invocation rules are identical to voice version.

STEP 5 – Opening logic follows the same session-state branching
as voice, but phrased as text messages, not spoken sentences.

Output format: Clean markdown. Same section header names as voice.
Write example messages in the target language specified in the answers.

--- ANSWERS START ---

[PASTE QUESTIONNAIRE ANSWERS HERE]

--- ANSWERS END ---
```